

AhamX Bodhi

Where I Becomes Infinite

Prompt Chaining

Module 2.3: Prompt Engineering | Prof. Sashikumaar Ganesan |
IISc Bengaluru



Session Overview

What You Will Learn

- What prompt chaining is and why it is needed
- Sequential and conditional chain patterns
- How chains decompose complex tasks
- Prompt chaining in LangChain and LangGraph

Session Details

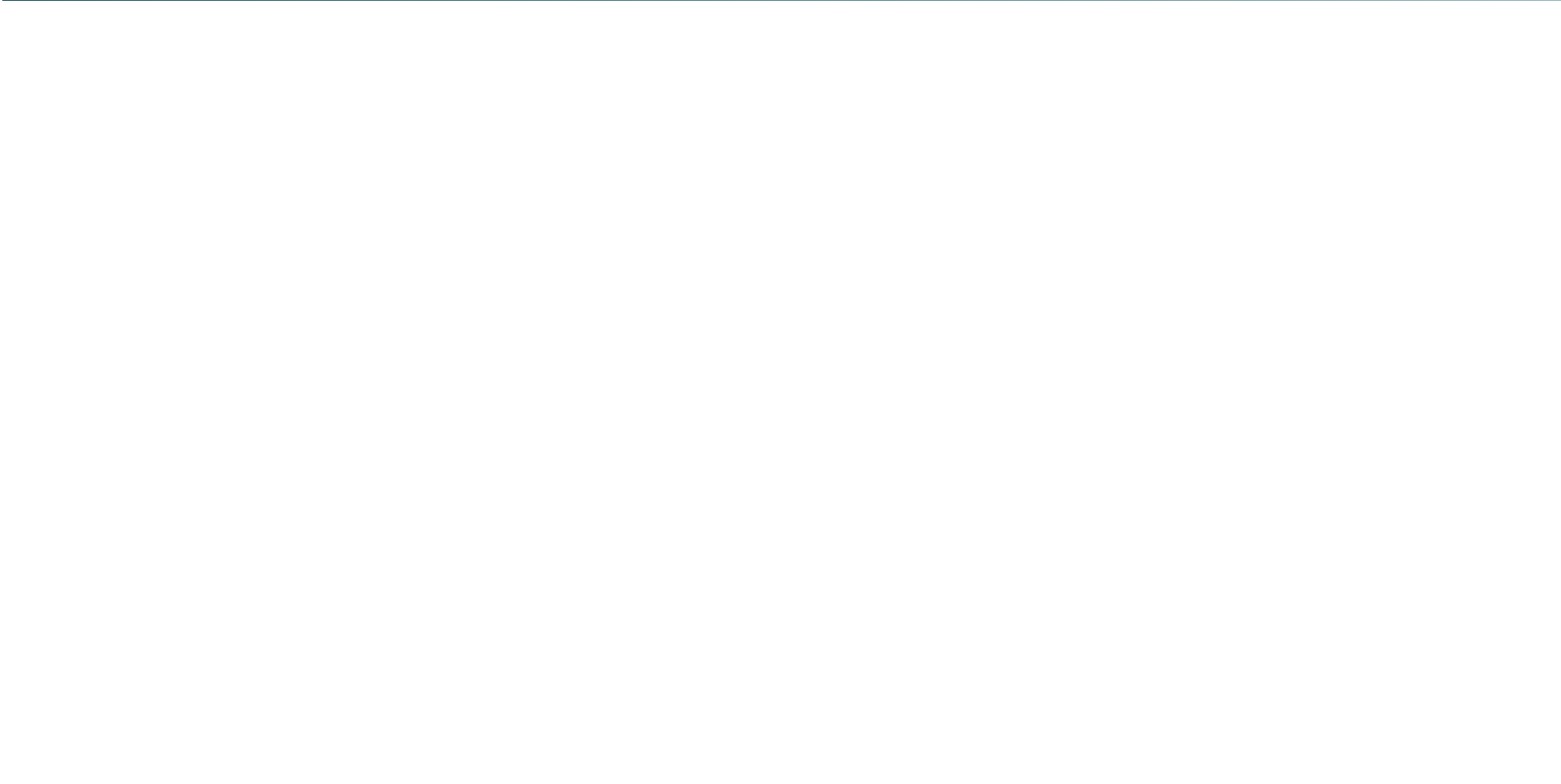
- **Duration:** 9 minutes
- **Bloom's Level:** B3 — Apply
- **Difficulty:** L2 — Intermediate
- **Module:** 2.3 Prompt Engineering
- **Prereq:** Prompt Templates and Patterns



Learning Objectives

By the End of This Session You Will Be Able To

- **Define** prompt chaining and explain when it is necessary
- **Design** sequential chains that pass outputs between LLM calls
- **Distinguish** sequential, conditional, and parallel chain patterns
- **Implement** basic chains using LangChain's LCEL syntax





Prompt Chaining: Definition

Core Definition

Prompt chaining is the practice of decomposing a complex task into a sequence of simpler sub-tasks, each handled by a separate LLM call. The output of one call becomes part of the input prompt for the next call, forming a chain.

Why Chains are Necessary

- Single prompts struggle with tasks requiring many independent reasoning steps
- Chains allow intermediate outputs to be verified, filtered, or transformed
- Each chain step can use a different prompt, model, or tool
- Enables complex workflows impossible with a single prompt call



Why a Single Prompt Is Not Always Enough

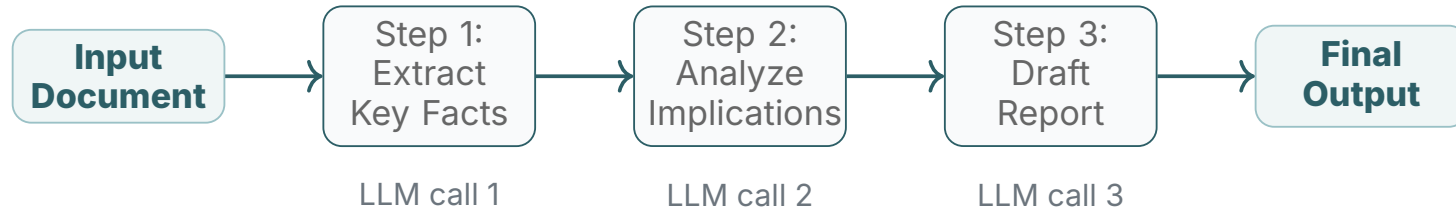
Limitations of Single-Call Prompting

- **Context window overflow:** Complex tasks generate too much intermediate content
- **Error accumulation:** One mistake in a long reasoning chain degrades all subsequent steps
- **Mixed concerns:** Extraction + analysis + formatting are each separate skills
- **No intermediate validation:** Cannot check or filter outputs between steps

Prompt chaining solves these by making each concern a separate, testable LLM call.



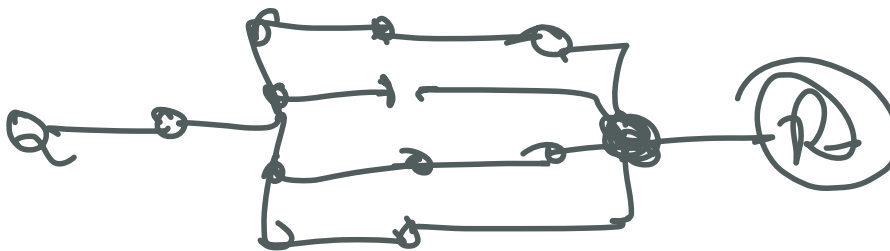
Sequential Chain: Step-by-Step Flow



Chain Properties

- Each step receives the output of the previous step as part of its input
- Steps can use different prompts, models, or even non-LLM tools (search, code execution)
- Intermediate outputs can be stored, logged, and validated

Three Core Chain Patterns



Sequential Chain

Steps execute in order. Output n feeds into input $n + 1$. Best for: multi-step text processing (extract $\rightarrow$ analyze $\rightarrow$ format).

Parallel Chain

Multiple LLM calls run simultaneously; results are merged. Best for: research tasks requiring multiple independent analyses combined into one report.

Conditional Chain

A routing step classifies input and directs it to one of multiple chains. Best for: intent routing in customer service (billing vs. technical vs. general).

Map-Reduce Chain

Same prompt applied to many chunks (map); results summarized (reduce). Best for: summarizing long documents that exceed context window.



Prompt Chaining with LangChain LCEL

LangChain Expression Language (LCEL)

LCEL uses the `|` pipe operator to compose chains declaratively. Each component receives the output of the previous one.

```

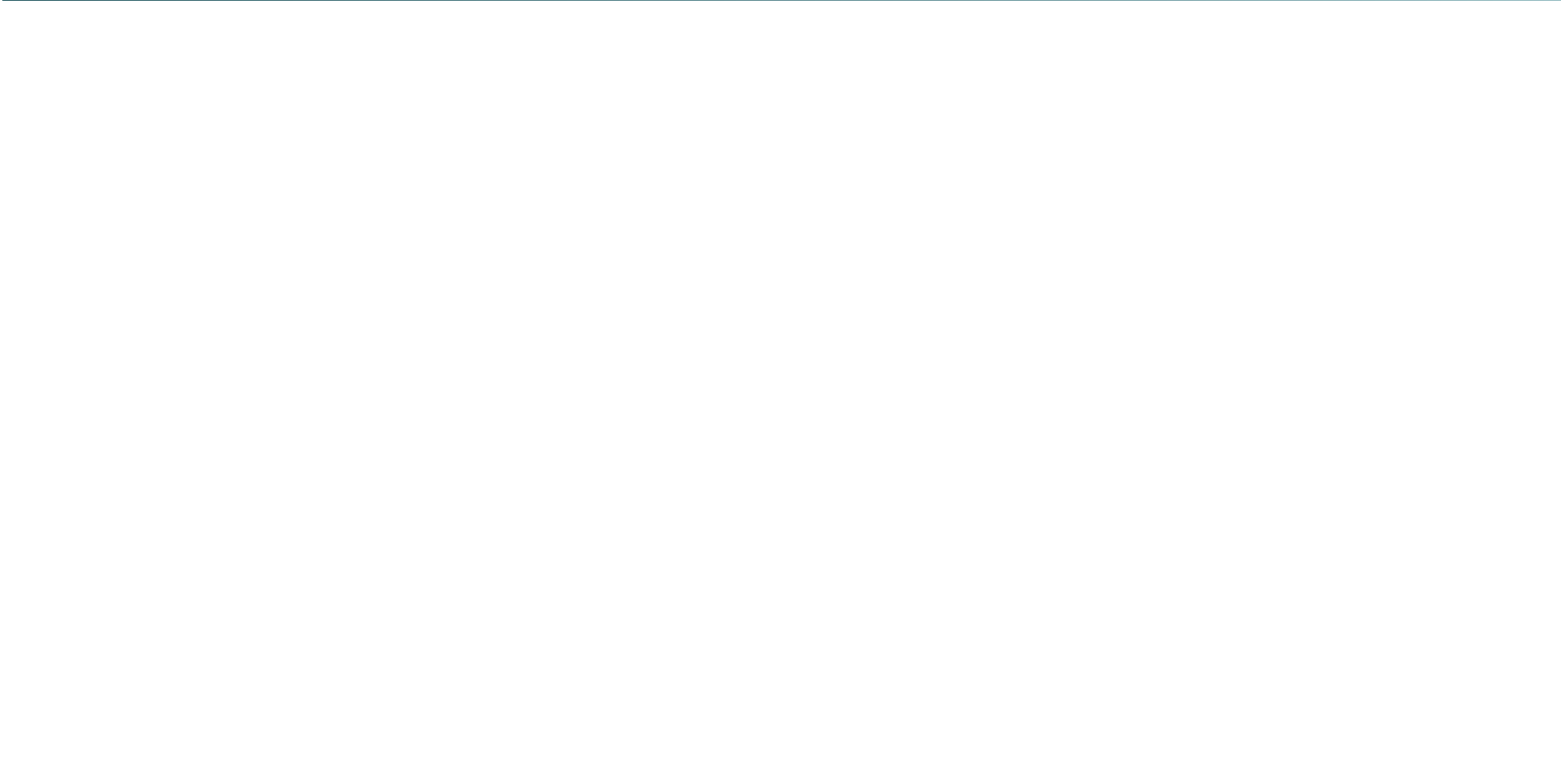
1 from langchain_openai import ChatOpenAI
2 from langchain_core.prompts import ChatPromptTemplate
3 from langchain_core.output_parsers import StrOutputParser
4
5 # Step 1: Extract key facts
6 extract_prompt = ChatPromptTemplate.from_template(
7     "Extract the 3 key facts from: {document}"
8 )
9 # Step 2: Summarize extracted facts
10 summary_prompt = ChatPromptTemplate.from_template(
11     "Write a one-paragraph summary of: {facts}"
12 )
13 model = ChatOpenAI(model="gpt-4o")
14 parser = StrOutputParser()

```

```

15
16 # Chain: document -> facts -> summary
17 chain = (extract_prompt | model | parser

```





Application: Multi-Stage Document Processing

Legal and Financial Document Workflows

Law firms and investment banks use prompt chains to process large documents: Step 1 extracts relevant clauses, Step 2 classifies risk level, Step 3 generates a remediation recommendation. Each step is auditable and can be reviewed by a human before proceeding.

Chain Benefits for Document Processing

- Handles documents exceeding single context window via map-reduce
- Each step can be tested and benchmarked independently
- Human review gates can be inserted between chain steps



Application: Automated Research Reports

Perplexity, Elicit, Consensus AI

AI research tools use parallel and sequential chains to produce comprehensive reports: Parallel chains search multiple sources simultaneously; a sequential chain synthesizes findings into a structured report with citations.

Research Chain Architecture

- **Step 1 (parallel):** Query multiple search engines and databases simultaneously
- **Step 2 (sequential):** Filter and deduplicate retrieved information
- **Step 3 (sequential):** Synthesize into structured report with section headings
- **Step 4:** Fact-check key claims against retrieved sources



Application: Agentic Task Automation

The Foundation of Agent Systems

Autonomous agents (LangGraph, AutoGen, CrewAI) are built on prompt chains where each step can call tools (APIs, code execution, search) and pass results to the next LLM call. Prompt chaining is the core mechanism that enables agents to complete multi-step real-world tasks.

Chain + Tool Use = Agents

When prompt chains include tool calls (web search, code execution, database queries) between LLM steps, you have the foundation of an agentic AI system.





Key Takeaways

Core Concepts to Remember

- Prompt chaining decomposes complex tasks into sequential LLM calls
- Output of each step feeds as input to the next step
- Four core patterns: sequential, conditional, parallel, map-reduce
- LangChain LCEL provides declarative chain composition with the `|` operator
- Prompt chaining is the foundation of all agentic AI systems

AhamX Bodhi

Where I Becomes Infinite